

Moon Landing

Yehor Panteliev

panteyeh@cvut.fit.cz

<https://github.com/wondab0y/lunar-lander-simulator>

May 24, 2026

1 Introduction

The goal of this project was to investigate a classic aerospace engineering problem - autonomous lunar landing. Given the fact that the spacecraft mass changes continuously during flight as fuel is consumed, we are forced to solve a nonlinear dynamical system while balancing several competing objectives such as fuel efficiency, landing safety, and control stability. We design and analyze a physics-based lunar landing simulator capable of modeling powered descent under realistic variable-mass rocket dynamics. It was implemented in Python and includes numerical integration methods for solving the required ODEs together with multiple autonomous landing strategies. Apart from manual control which allows user to play the game and attempt to land the aircraft safely, there are several autopilot systems, including a bang-bang controller inspired by optimal control theory, a classical PID controller, and reinforcement learning based controllers trained using Q-learning. An important part of the project was also studying how different control objectives influence the resulting landing behavior. To evaluate the robustness and overall performance of the implemented methods, Monte Carlo experiments with randomized initial conditions were performed and analyzed statistically. The final result is both an interactive simulation and an experimental framework for comparing different autonomous landing strategies under nonlinear rocket dynamics.

2 Constructed Equations

We are describing a 1-d lunar descent, so our model will remain relatively simple. The spacecraft state is represented by three time-dependent variables: altitude $h(t)$, vertical velocity $v(t)$, and spacecraft mass $m(t)$. The dynamics of the system are governed by the following ODEs:

$$\begin{aligned}\dot{h} &= v, \\ \dot{v} &= \frac{T}{m} - g, \\ \dot{m} &= -\frac{T}{v_e}.\end{aligned}$$

The first equation shows that the rate of change of altitude equals the current vertical velocity. The second equation follows directly from Newton's second law, the third equation models fuel consumption assuming constant exhaust velocity and continuous propellant flow. Additionally, we normalize engine thrust through a throttle parameter

$$u \in [0, 1],$$

such that:

$$T = uT_{\max}.$$

2.1 Bang-Bang Control and Variable-Mass Braking

Following a very simple idea that fuel is only consumed while the engine is firing, it makes sense for us to assume that it's efficient for us to keep the engine off as long as possible. Basically, we delay braking for as long as we can, then perform an aggressive landing burn. There are multiple real world cases that use this idea, the most known one being SpaceX Falcon 9. When Falcon 9 lands, it free-falls for most of descent, then performs a late "landing burn," often with engines near maximum thrust. Despite being very fuel efficient, it's also extremely unforgiving, thus unsafe for astronauts, which prevents it from being widely used. To simulate it, we have to find a formula for stopping distance, at which our aircraft will activate full thrust.

The initial implementation of the bang-bang controller used a simplified stopping-distance approximation based on constant acceleration:

$$d = \frac{v^2}{2a},$$

where

$$a = \frac{T_{\max}}{m} - g.$$

This approximation assumes that mass remains constant during braking. However, this assumption is not accurate during long burns, because the spacecraft mass decreases continuously as fuel is consumed. As a result, the stopping-distance estimate can produce incorrect ignition timing.

To improve this, the final bang-bang controller was based on the variable-mass rocket equation. Under full thrust, the mass flow rate is

$$\dot{m}_f = \frac{T_{\max}}{v_e}.$$

Therefore, after burning for time t , the remaining mass is

$$m_f = m_0 - \dot{m}_f t.$$

The velocity change due to rocket thrust is described by the Tsiolkovsky rocket equation:

$$\Delta v = v_e \ln \left(\frac{m_0}{m_f} \right).$$

Including the effect of lunar gravity during the burn gives

$$v_f = v_0 + v_e \ln \left(\frac{m_0}{m_f} \right) - gt.$$

This equation predicts the velocity after burning for time t , starting with initial velocity v_0 , initial mass m_0 , and final mass m_f . The logarithmic term appears because the spacecraft acceleration changes as mass decreases.

To predict where the spacecraft will be when braking finishes, the velocity equation is integrated over time:

$$h(t) = h_0 + \int_0^t v(\tau) d\tau.$$

This gives the altitude after a full-thrust burn:

$$h_f = h_0 + v_0 t + v_e I(m_0, m_f) - \frac{1}{2} g t^2,$$

where

$$I(m_0, m_f) = \frac{m_0 - m_f \left(\ln \left(\frac{m_0}{m_f} \right) + 1 \right)}{\dot{m}_f}.$$

The bang-bang controller uses this formula to determine the latest safe ignition point. For each state of the spacecraft, it checks whether a full-thrust burn starting at the current moment can reduce the vertical velocity to zero before reaching the surface. The burn duration is found numerically using binary search for the condition

$$v_f \approx 0.$$

Then the corresponding stopping altitude h_f is evaluated. If

$$h_f \leq 0,$$

the spacecraft must begin full-thrust braking immediately. Otherwise, it continues free fall.

Thus, the final bang-bang strategy has the form

$$u(t) = \begin{cases} 0, & \text{before the ignition point,} \\ 1, & \text{after the ignition point.} \end{cases}$$

This makes the controller fuel-efficient, because it keeps the engine off for as long as possible and then uses maximum thrust for the final braking phase.

2.2 PID Control

Despite being extremely efficient, bang-bang control has an obvious disadvantage of being very hard to safely execute in real life. Thus, most spaceships normally count on smoother landing strategies. In order to obtain and simulate one, we develop a PID controller. Unlike bang-bang control, the PID controller continuously adjusts throttle based on the difference between the current velocity and a desired target velocity.

The velocity error is defined as

$$e(t) = v_{\text{target}}(t) - v(t).$$

The PID control law is

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de}{dt},$$

where K_p , K_i , and K_d are the proportional, integral, and derivative gains.

The target velocity was chosen as a function of altitude:

$$v_{\text{target}}(h) = -k\sqrt{h}.$$

This choice allows faster descent at high altitude while forcing the spacecraft to slow down naturally as it approaches the lunar surface. In contrast to bang-bang control, PID control is smoother and generally safer, but it may consume more fuel because it applies corrective thrust earlier and more continuously.

2.3 Reinforcement Learning Formulation

Reinforcement learning controllers were also implemented using tabular Q-learning. The state was represented by a discretized version of altitude, velocity, and remaining fuel. The action space consisted of a finite set of throttle levels:

$$u \in \{0, 0.25, 0.5, 0.75, 1\}.$$

The Q-learning update rule is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where s is the current state, a is the selected action, r is the reward, s' is the next state, α is the learning rate, and γ is the discount factor.

Two reward structures were tested. The first reward function emphasized fuel efficiency, encouraging the agent to use thrust only when necessary. The second reward function added a penalty for abrupt throttle changes, encouraging smoother control. This made it possible to compare whether a learned controller would behave more like bang-bang control under fuel efficiency objectives, or more like PID control under smoothness-based objectives.

3 Methods/Procedures/Algorithms

The project developed a physics-based simulation of autonomous lunar landing. The implementation was divided into several independent components responsible for physical modeling, numerical integration, control systems, reinforcement learning, and experimental analysis.

The physical model was implemented using a system of ODEs describing altitude, vertical velocity, and spacecraft mass. All theory used can be found in Constructed Equation subsection.

We implemented several numerical integration methods, namely explicit Euler method, RK2 and RK4. As expected, after experimentation RK4 was selected as the primary integration method due to better numerical stability.

The simulator supports both manual control and multiple autonomous landing strategies. Manual control allows the user to interactively pilot the spacecraft by adjusting engine throttle in real time. Telemetry and live plots displaying altitude, velocity, fuel level, and throttle were also integrated directly into the simulation to allow obtaining information about the process in real time.

The first landing strategy implemented was a bang-bang controller which focuses on optimal fuel use. All the underlying theory can be found in 2.1.

A classical PID controller was also implemented. Similarly, all the theory was described in 2.2, in this section we just create it in Python.

In addition to manually designed controllers, reinforcement learning based controllers were implemented using tabular Q-learning. After using formulas from 2.3 and optimizing parameters, we were left with 2 working RL-agents: one is focused on fuel-efficiency, the other - on smoothness of our landing.

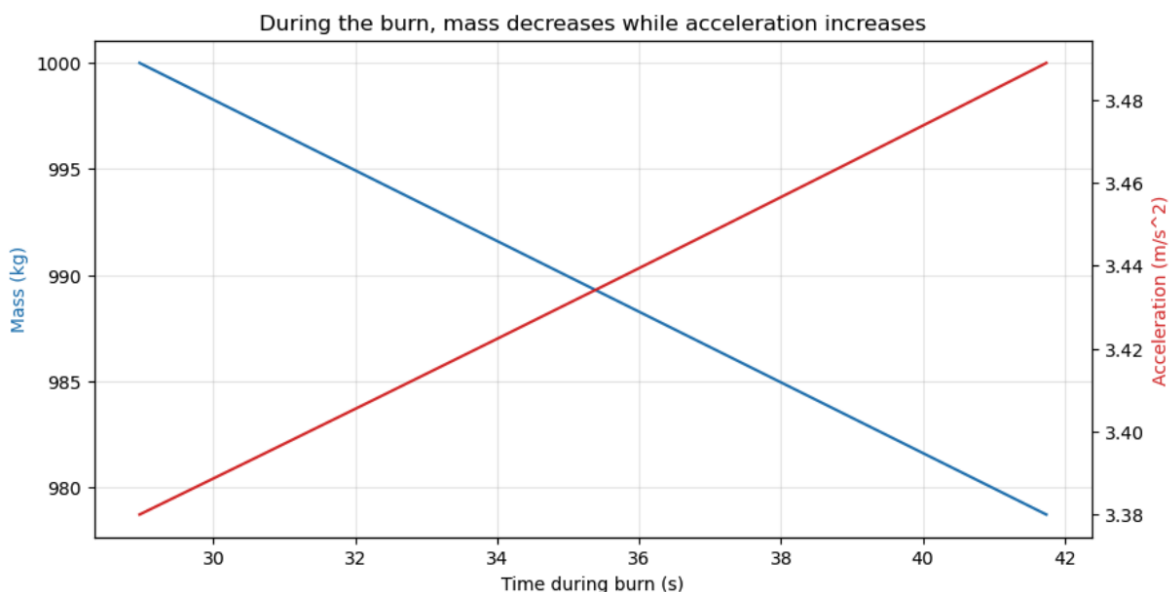
To evaluate the robustness and performance of the implemented controllers, Monte Carlo experiments were performed using randomized initial conditions. Multiple autonomous landing strategies were compared statistically using metrics such as success rate, fuel consumption, landing velocity, and descent duration. Additional offline visualizations and plots were generated using Jupyter notebooks and Matplotlib in order to analyze and compare controller behavior in greater detail.

4 Conclusions

In this section we describe the results obtained from experimenting with designed simulation. All source code and generated charts are available in Github repo of the project. To get enough data for conclusions, we run a huge 5000 trial Monte Carlo experiment. Obtained information will be reused throughout all visualizations.

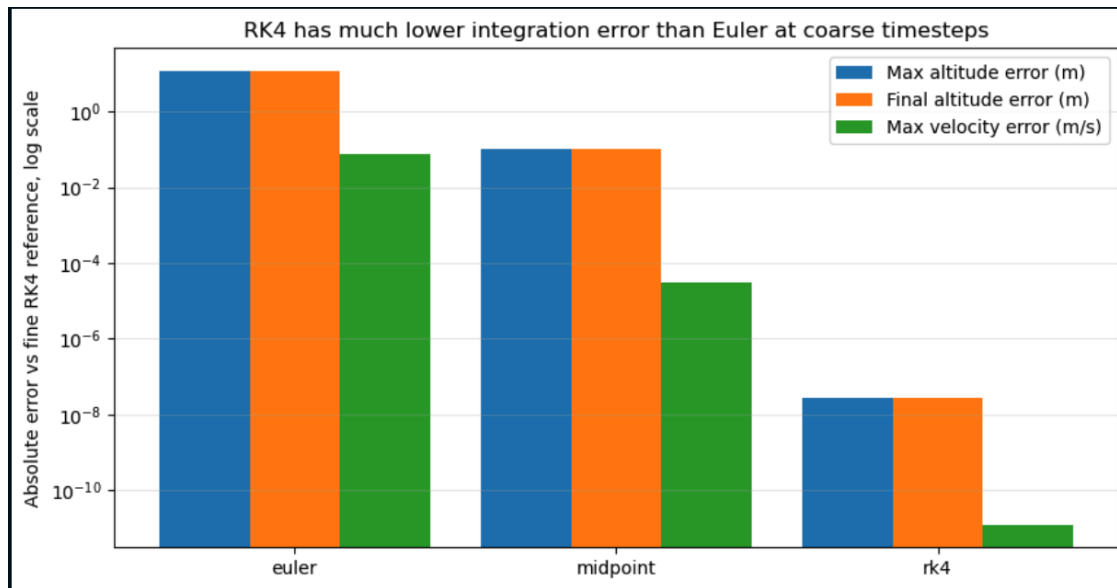
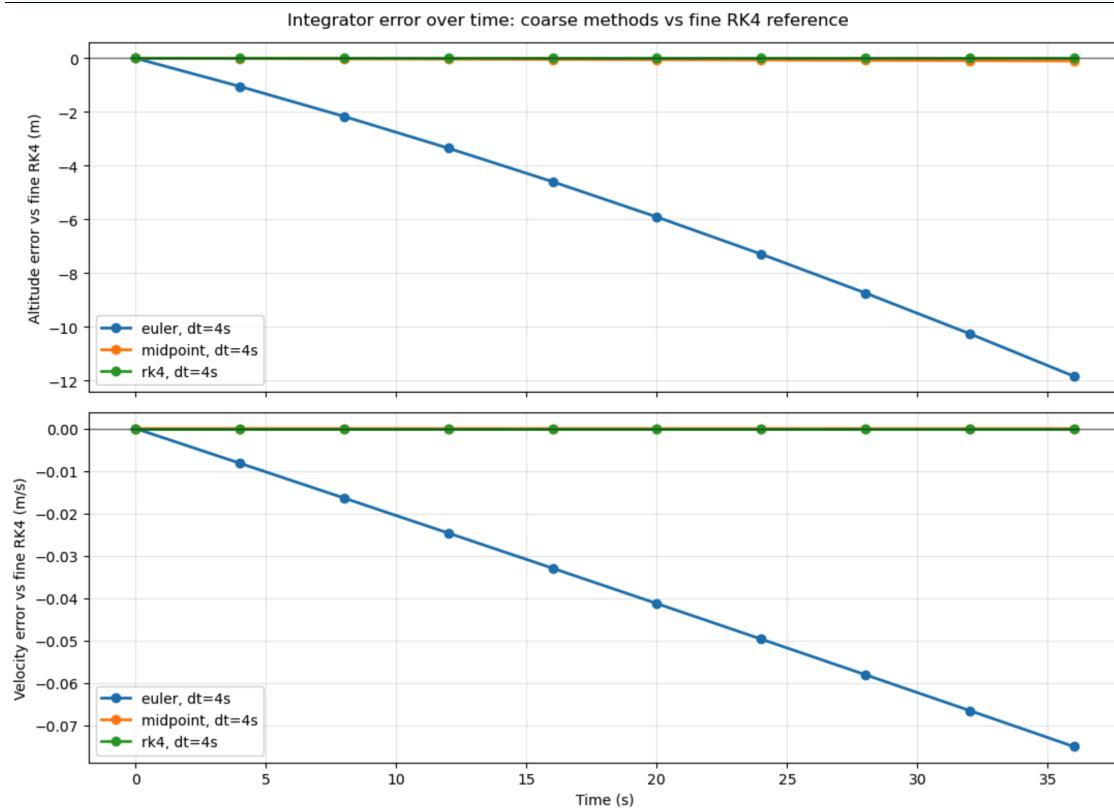
4.1 Variable Mass Makes Landing Nonlinear

The spacecraft mass changes continuously as fuel burns. Because thrust acceleration is T / m , the same thrust produces larger acceleration as mass decreases, which is shown on graphs below:



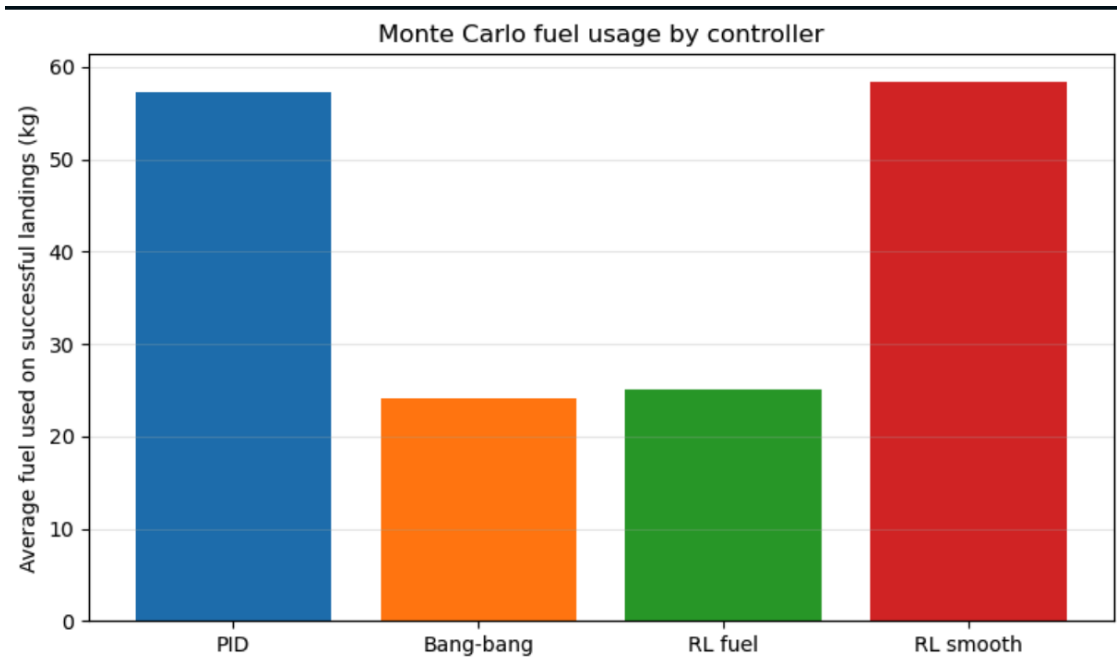
4.2 Numerical Integration Choice Affects Simulation Quality

With the normal simulation timestep ($DT = 0.02$), Euler, midpoint/RK2, and RK4 visually overlap as they give extremely similar values. To make integration error visible, this section uses a fixed-throttle descent problem with a bigger timestep and compares each method against a fine-step RK4 reference. This isolates numerical integration quality from controller behavior.



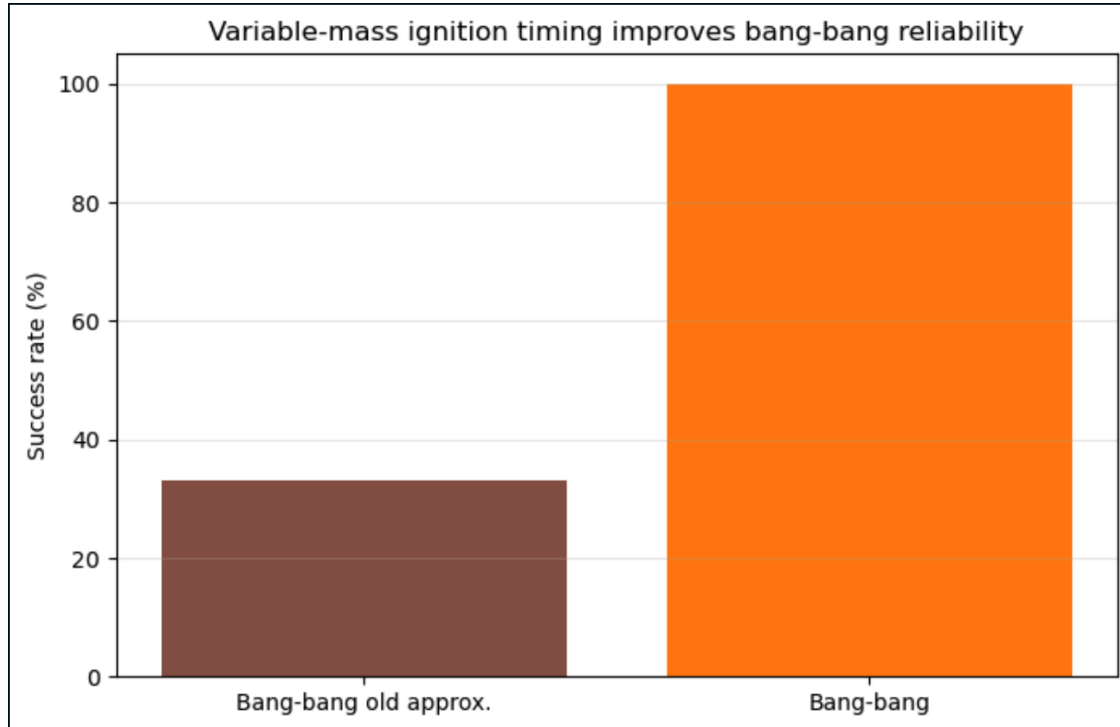
4.3 Bang-Bang Uses The Least Fuel

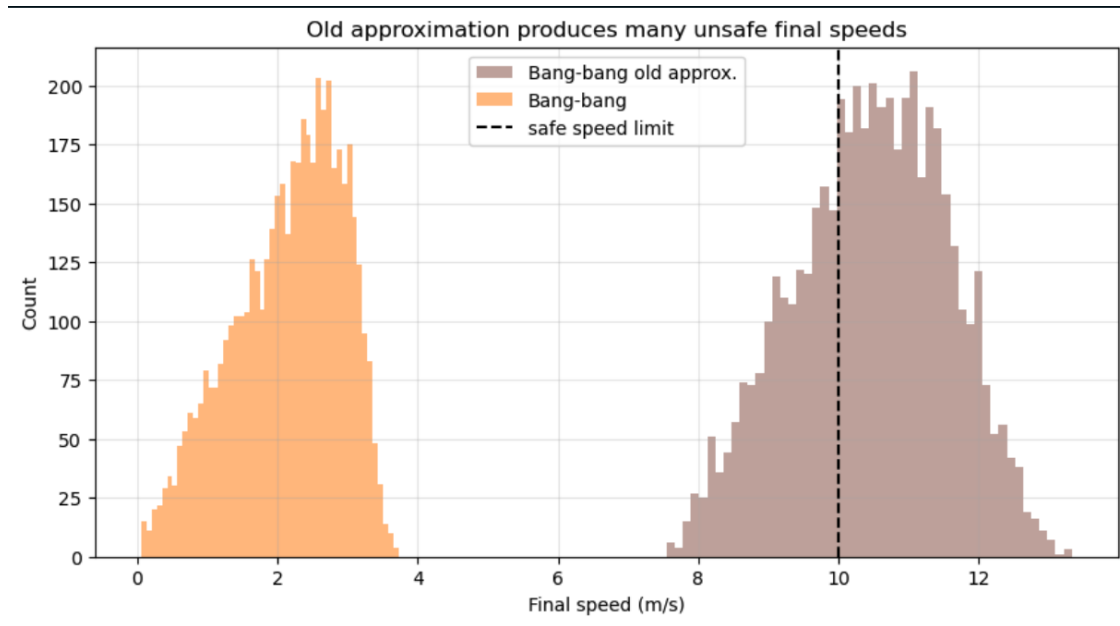
Bang-bang keeps the engine off as long as possible, so it should minimize fuel relative to smoother feedback controllers. RL-fuel, which demonstrates similar behaviour, should also be fuel efficient.



4.4 Constant Acceleration Bang-Bang Is Not Accurate Enough

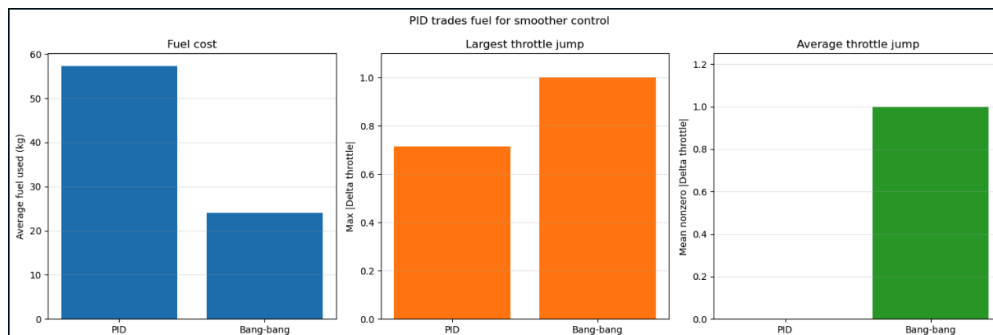
The old bang-bang trigger assumed constant acceleration. The current controller predicts braking time using rocket-equation terms, which improves ignition timing.





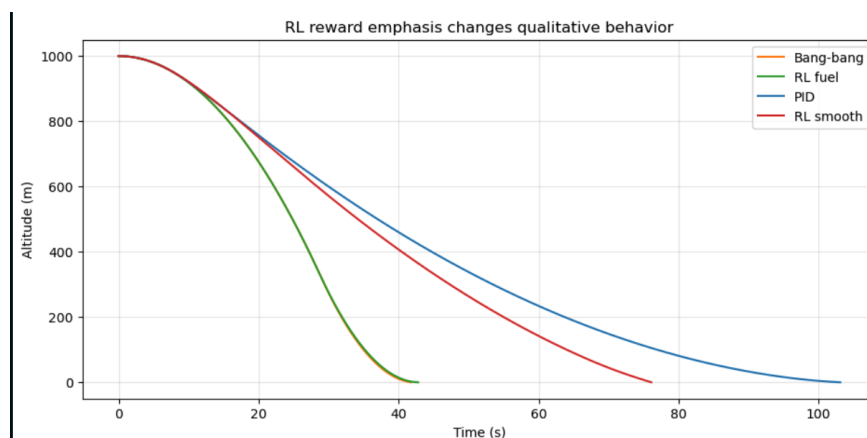
4.5 PID Is Smoother but Uses More Fuel Than Bang-Bang

PID prioritizes smooth descent and low throttle change rate over efficient fuel use



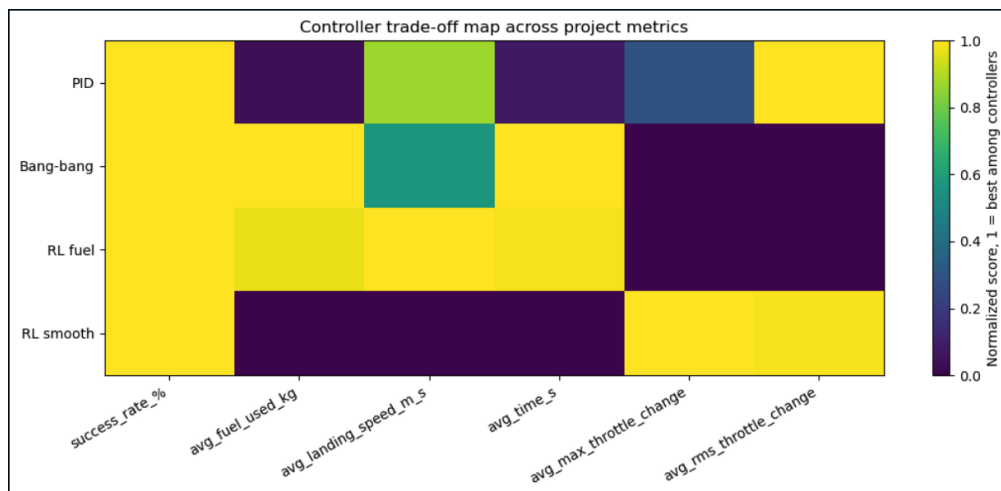
4.6 RL Reward Structure Determines Learned Behavior

RL fuel behaves like fuel-efficient bang-bang control. RL smooth behaves more like PID control.



4.7 A Combined Framework Shows Controller Trade-Offs

The final comparison shows advantages and disadvantages of each controller type



P.S. As it was already mentioned, the game/simulation can be installed following README instructions in Github repo of this project (link provided in the title). In case there are any issues installing it, I will also provide short video demonstrations of its main capabilities in project folder.